

EDGE INTELLIGENCE

LAB - 7

25MML0038

SUBHASHINI M

```
Generate Code Markdown Generate Jupyter Cell with Cite Run AI Clear All Outputs Outline
D:\>
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions

# Load pretrained MobileNetV2 model
model = MobileNetV2(weights='imagenet')

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Decide step size to evenly sample frames
    if total_frames > 0:
        step = max(1, total_frames // max_frames)
    else:
        step = 1

    count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)

        count += 1

        if len(frames) >= max_frames:
            break

    cap.release()
    print("Extracted frames:", len(frames))
    return np.array(frames)

def classify_video(video_path):
    frames = extract_frames(video_path)
```

```
def classify_video(video_path):
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames extracted!")
        return

    # Preprocess frames
    frames = preprocess_input(frames)

    # Predict
    predictions = model.predict(frames)

    # Average predictions across frames
    avg_pred = np.mean(predictions, axis=0)

    # Decode top 5 predictions
    top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)

    print("\nTop 5 Predictions:\n")
    for pred in top_pred[0]:
        print(pred[1], ":", round(pred[2] * 100, 2), "%")

    print("\nFinal Prediction:", top_pred[0][0][1])
```

```
# Your video path in Colab
video_path = "/content/real_world_demo_video.mp4"

classify_video(video_path)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels/1.0_224.h5
14536120/14536120 — 0s 0us/step
Extracted frames: 20
1/1 — 2s 7s/step
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 — 0s 0us/step

Top 5 Predictions:

digital_clock : 63.84 %
traffic_light : 9.13 %
switch : 1.43 %
computer_keyboard : 1.32 %
matchstick : 0.85 %

Final Prediction: digital_clock
```

```
def analyze_and_show_video(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    # 1. Get Dedicated Video Data (Metadata)
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    original_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    original_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    print(f"--- DEDICATED VIDEO DATA ---")
    print(f"Resolution: (original_w)x(original_h) | FPS: {fps:.2f} | Total Frames: (total_frames)\n")

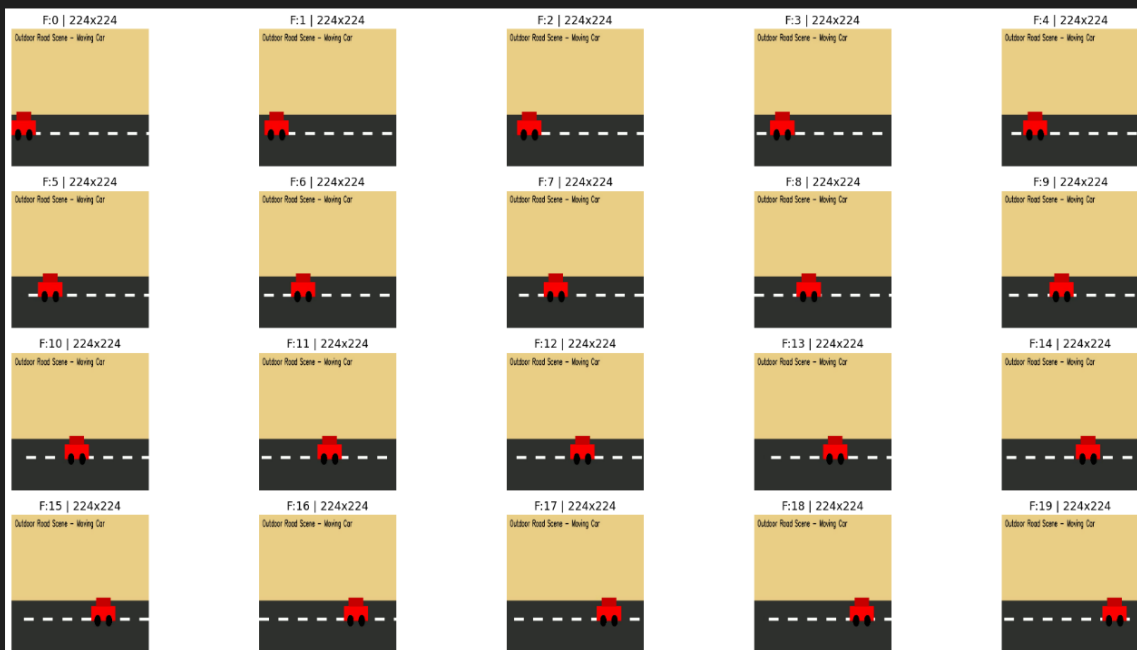
    # 2. Extract Frames
    frames = []
    step = max(1, total_frames // max_frames)
    for i in range(max_frames):
        cap.set(cv2.CAP_PROP_POS_FRAMES, i * step)
        ret, frame = cap.read()
        if not ret: break
        # Resize to 224x224 as per your MobileNet requirement
        frame = cv2.resize(frame, (224, 224))
        frames.append(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    cap.release()

    # 3. Show Frames with Dimensions (Your Style)
    plt.figure(figsize=(20, 10))
    for i in range(len(frames)):
        h, w, _ = frames[i].shape
        plt.subplot(4, 5, i + 1)
        plt.imshow(frames[i])
        plt.title(f"F:{i} | {w}x{h}")
        plt.axis('off')
    plt.tight_layout()
    plt.show()

    # 4. Display Pixel Matrix for the Last Captured Frame
    print(f"--- PIXEL MATRIX DATA (Frame {len(frames)-1}) ---")
    # Show a 5x5 slice of the RGB values
    print(frames[-1][0:5, 0:5])

# Run the analysis
analyze_and_show_video(video_path)
```

```
--- DEDICATED VIDEO DATA ---
Resolution: 854x480 | FPS: 24.00 | Total Frames: 192
```



```
--- PIXEL MATRIX DATA (Frame 19) ---
```

```
[[[233 206 133]
 [233 206 133]
 [233 206 133]
 [233 206 133]
 [233 206 133]]

 [[233 206 133]
 [233 206 133]
 [233 206 133]
 [233 206 133]
 [233 206 133]]

 [[233 206 133]
 [233 206 133]
 [233 206 133]
 [233 206 133]
 [233 206 133]]

 ...
 [[233 206 133]
 [233 206 133]
 [233 206 133]
 [233 206 133]
 [233 206 133]]]
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#).

Step 1: Import the required libraries such as OpenCV for video processing and Matplotlib for visualization.

Step 2: Load the video file using video capture functionality to access the frames.

Step 3: Verify that the video file is successfully opened to prevent runtime errors.

Step 4: Read the video frame by frame using a loop structure.

Step 5: Select multiple frames at fixed intervals (for example, every 30th frame) instead of processing all frames to reduce redundancy.

Step 6: Convert the frame color format from BGR to RGB for proper visualization.

Step 7: Store the selected frames in a list for organized access and further processing.

Step 8: Display the extracted frames in a structured grid format for better comparison and analysis.

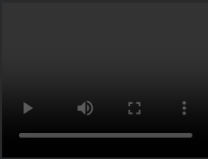
Step 9: Release the video capture object to free system memory and properly close the video file.

TASK 2

```
[ ] + Code + Text
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from glob import glob
import IPython.display as ipd
from tqdm.notebook import tqdm

[ ] #import subprocess

[ ] ipd.Video("026c7465-309f6d33.mp4",width = 200)
```



```
[ ] |pip install opencv-python
```

Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)

```
[ ] # Load in video capture
import cv2
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
```

```
[ ] # Total number of frames in video
cap.get(cv2.CAP_PROP_FRAME_COUNT)
```

2398.0

```
[ ] # Video height and width
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
print(f'Height {height}, Width {width}')
```

Height 720.0, Width 1280.0

```
[ ] cap.release()
```

Pull the image from the first frame

```
[ ] cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
ret, img = cap.read() #will return TRUE everytime we call this function till we reach the end of the video
print(f'Returned {ret} and img of shape {img.shape}')
```

Returned True and img of shape (720, 1280, 3)

Loading the First Image

```
[ ] plt.imshow(img) #opencv loads color in BGR format, matplotlib lib expects RGB format
```

<matplotlib.image.AxesImage at 0x7faad2e85be0>



```
[ ] ## Helper function for plotting opencv images in notebook
def display_cv2_img(img, figsize=(10, 10)):
    img_ = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    fig, ax = plt.subplots(figsize=figsize)
    ax.imshow(img_)
    ax.axis("off")
```

```
[1]  ## Helper function for plotting opencv images in notebook
def display_cv2_img(img, figsize=(10, 10)):
    img_ = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    fig, ax = plt.subplots(figsize=figsize)
    ax.imshow(img_)
    ax.axis("off")

[1]  display_cv2_img(img)

...

```

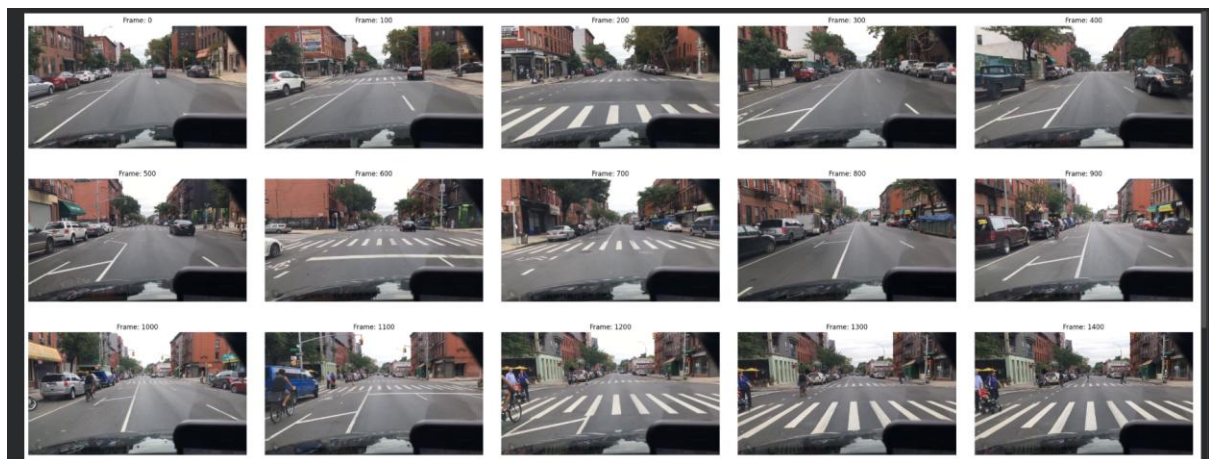
Display Multiple Frames from the Video

```
[1]  import cv2
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()

cap = cv2.VideoCapture("026c7465-309fd33.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

img_idx = 0
for frame in range(n_frames):
    ret, img = cap.read()
    if ret == False:
        break
    if frame % 100 == 0:
        axs[img_idx].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        axs[img_idx].set_title(f'Frame: {frame}')
        axs[img_idx].axis('off')
        img_idx += 1

plt.tight_layout()
plt.show()
cap.release()
```



```

1 labels = pd.read_csv('mot_labels.csv', low_memory=False)
  ##labels.head()
  #video_labels = (labels.query('videoName == "026c7465-309fed33").reset_index(drop=True).copy())
  #video_labels["video_frame"] = (video_labels["frameIndex"] * 11.9).round().astype("int") #video is at 60Hz but the labellers did only 5 Hz

2
3 #print(video_labels.shape)
4
5 (0, 14)
6
7 print(labels["category"].value_counts())
8
9 category
10 car          394391
11 pedestrian    67862
12 truck         22812
13 bus           10694
14 bicycle       3823
15 other vehicle 3756
16 rider         3307
17 motorcycle    2277
18 other person   782
19 trailer        376
20 train         156
21 Name: count, dtype: int64
22
23 cap = cv2.VideoCapture("026c7465-309fed33.mp4")
24 n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
25
26 img_idx = 0
27 for frame in range(n_frames):
28     ret, img = cap.read()
29     if ret == False:
30         break
31     if frame == 1035:
32         break
33     cap.release()
34
35 display_cv2_img(img)
36
37 display_cv2_img(img)
38
39 
```

Step 1: Imported the necessary libraries such as OpenCV to handle video processing tasks.

Step 2: Specified the path of the video file that needs to be processed.

Step 3: Created a video capture object to load and access the video file.

Step 4: Checked whether the video file was successfully opened to ensure there were no loading errors.

Step 5: Read the video frame by frame using a loop structure.

Step 6: Displayed each frame of the video for visualization and verification.

Step 7: Controlled the playback speed using delay functions to simulate real-time video display.

Step 8: Added a condition to stop the video playback when a specific key was pressed.

Step 9: Released the video capture object after processing to free system resources.

Step 10: Closed all display windows properly to complete the program execution safely.

EDGE INTELLIGENCE

LAB - 8

25MML0038

SUBHASHINI M

```
[ ]
import cv2
import matplotlib.pyplot as plt

# 1. Load video and extract 1 frame
cap = cv2.VideoCapture('/content/026c7465-309f6d33.mp4')
ret, frame = cap.read()

if ret:
    # 2. Convert to Grayscale
    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 3. PRINT NUMERIC MATRICES (Top-Left 3x3 section for comparison)
    # Original is 3D (BGR), Grayscale is 2D (Single Intensity)
    print("--- ORIGINAL BGR MATRIX (3x3 Sample) ---")
    print(frame[0:3, 0:3])

    print("\n--- GRAYSCALE MATRIX (3x3 Sample) ---")
    print(gray_frame[0:3, 0:3])

    # 4. PRINT/VISUALISE BOTH IMAGES SIDE-BY-SIDE
    plt.figure(figsize=(12, 6))

    # Left: Original Color (Convert BGR to RGB for Matplotlib)
    plt.subplot(1, 2, 1)
    plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    plt.title("Original Color Frame")
    plt.axis('off')

    # Right: Grayscale
    plt.subplot(1, 2, 2)
    plt.imshow(gray_frame, cmap='gray')
    plt.title("Converted Grayscale Frame")
    plt.axis('off')
```

--- ORIGINAL BGR MATRIX (3x3 Sample) ---

```
[[[54 67 61]
   [54 67 61]
   [54 67 61]]
```

```
[[[54 67 61]
   [54 67 61]
   [54 67 61]]
```

```
[[[51 64 58]
   [51 64 58]
   [51 64 58]]]
```

--- GRAYSCALE MATRIX (3x3 Sample) ---

```
[[[64 64 64]
   [64 64 64]
   [61 61 61]]]
```

Original Color Frame



Converted Grayscale Frame



```
import matplotlib.pyplot as plt
```

```
# 1. Load the video and extract the first frame
```

```
cap = cv2.VideoCapture('/content/826c7465-389f6d33.mp4')
ret, frame = cap.read()
```

```
if ret:
```

```
    # 2. Convert to Grayscale (Pre-processing)
```

```
    # Edge detection works best on single-channel intensity images
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
    # 3. Apply Gaussian Blur
```

```
    # Highly recommended to reduce noise before detecting edges
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
    # 4. Apply Canny Edge Detection
```

```
    # threshold1 (100): Lower bound for hysteresis thresholding
    # threshold2 (200): Upper bound for hysteresis thresholding
    edges = cv2.Canny(blurred, 100, 200)
```

```
    # 5. Display the Results
```

```
    plt.figure(figsize=(10, 5))
```

```
    plt.subplot(1, 2, 1)
```

```
    plt.title("Original Grayscale")
```

```
    plt.imshow(gray, cmap='gray')
```

```
    plt.axis('off')
```

```
    plt.subplot(1, 2, 2)
```

```
    plt.title("Canny Edges")
```

```
    plt.imshow(edges, cmap='gray')
```

```
    plt.axis('off')
```

```
    plt.show()
```

```
else:
```

```
    print("Error: Could not extract frame.")
```

```
cap.release()
```

Original Grayscale



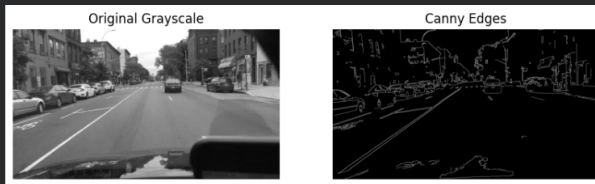
Canny Edges



```

plt.imshow(gray)
plt.show()
else:
    print("Error: Could not extract frame.")
cap.release()

```



```

import cv2
import numpy as np
from google.colab.patches import cv2_imshow

# 1. Load the video
video_path = '/content/026c7465-309f6d33.mp4'
cap = cv2.VideoCapture(video_path)
ret, frame = cap.read()

if ret:
    # 2. Convert to Grayscale first (optional, but clearer for matrices)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 3. CALCULATE THE NEGATIVE
    # Math: Negative = 255 - Original
    # We use cv2.bitwise_not for the fastest performance
    negative_img = cv2.bitwise_not(gray)

    # 4. PRINT PIXEL MATRICES (Top-Left 5x5 area)
    print("... ORIGINAL GRAYSCALE PIXELS ...")
    print(gray[0:5, 0:5])

```

```

... ORIGINAL GRAYSCALE PIXELS ...
[[64 64 64 64 64]
 [64 64 64 64 64]
 [61 61 61 61 61]
 [60 60 60 60 60]
 [59 59 58 58 57]]

```

```

... NEGATIVE IMAGE PIXELS (255 - Original) ...
[[191 191 191 191 191]
 [191 191 191 191 191]
 [194 194 194 194 194]
 [195 195 195 195 195]
 [196 196 197 197 198]]

```

```

... Visual: Original vs Negative ...

```



bles Terminal

9:51PM Python 3

EDGE INTELLIGENCE-09

NUMBER PLATE DETECTION PROJECT

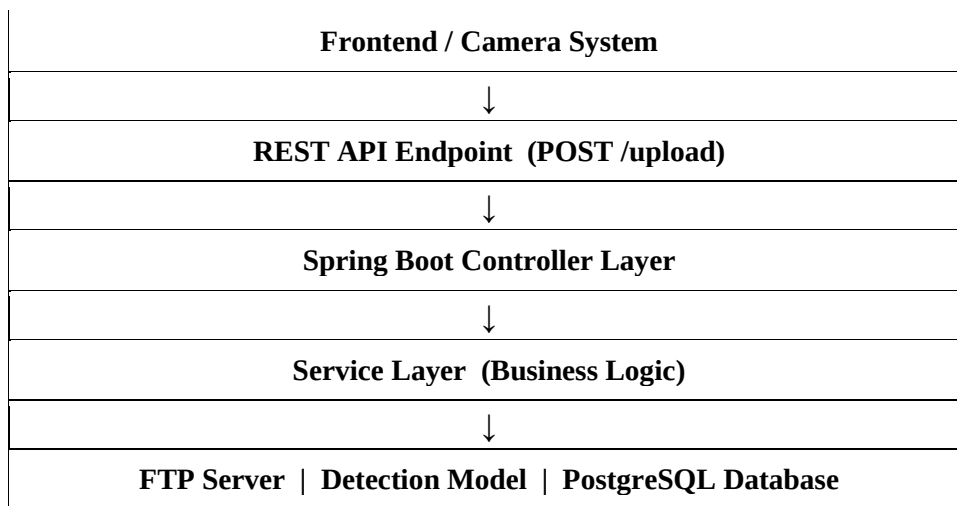
SUBHASHINI M
25MML0038

1. System Overview

The backend of the Number Plate Detection system is developed using Spring Boot with Java, structured around the MVC (Model-View-Controller) design pattern. Communication between the frontend and backend is handled through REST APIs, which serve as the primary entry points for all incoming client requests.

When a user captures or uploads a vehicle image from the frontend application, the image is transmitted to the backend via an upload API. The controller layer receives the request, validates it, and forwards it to the service layer for further processing.

System Overview Flow

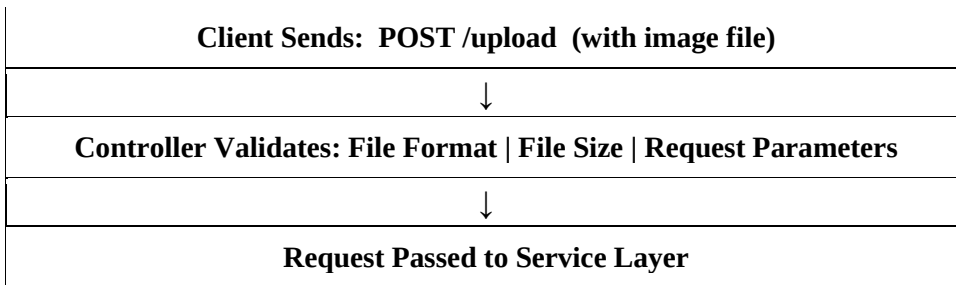


2. API Layer — Spring Boot Controller

Spring Boot provides RESTful API endpoints to receive and handle client requests. The Controller layer is responsible for the following:

- Receiving the uploaded image from the frontend
- Validating file format, file size, and request parameters
- Forwarding the validated request to the service layer

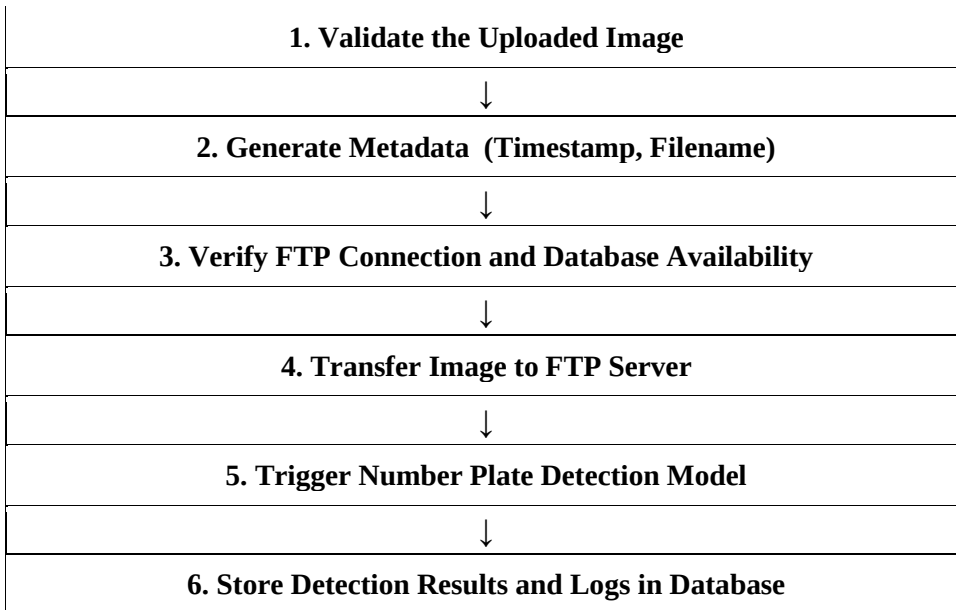
API Request Flow



3. Service Layer Processing

The Service class contains the core business logic of the application. After receiving a validated image request, the service layer carries out the following operations in order:

Service Layer Processing Flow

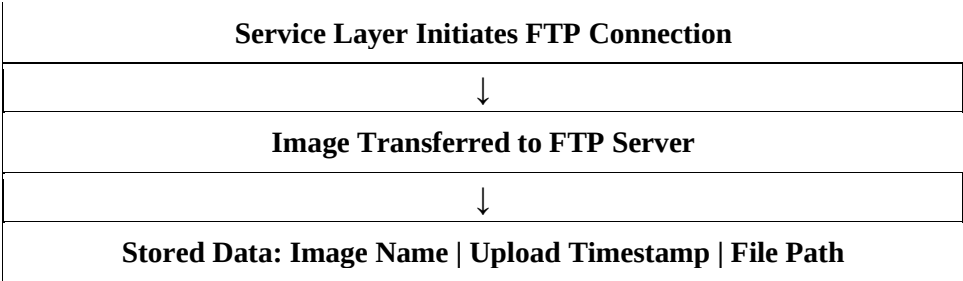


4. Image Storage — FTP Server

Once the image has been validated, it is transferred to a centralized FTP server for storage. This approach provides the following benefits:

- Centralized and secure storage of all uploaded vehicle images
- Reduced load on the main application server
- Efficient file management and easy accessibility for downstream processing

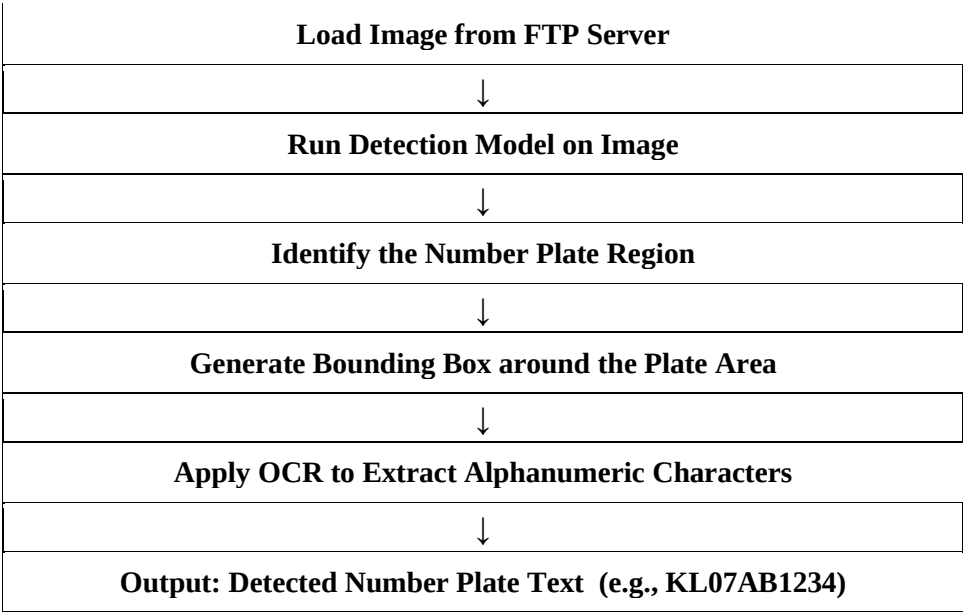
FTP Storage Workflow



5. Number Plate Detection — ML Model and OCR

Once the image is stored in the FTP server, the detection pipeline is triggered. The machine learning model processes the image and extracts the vehicle number plate text through the following steps:

Detection Pipeline Flow



6. Database Storage — PostgreSQL

All detection results and system activity logs are stored in a PostgreSQL relational database. The backend uses Hibernate ORM to manage all database interactions, ensuring reliable and efficient SQL operations. The following information is recorded for every request:

Database Fields

Field	Description
filename	Name of the uploaded image file
upload_time	Timestamp of the upload
detected_plate	Extracted number plate text
processing_status	Success or Failure
server_logs	System activity logs

7. Logging and Monitoring

A logging mechanism is implemented to track and monitor all system activities at each stage of processing. Logs are generated for the following events:

- API request received
- Image successfully uploaded to FTP server
- Detection model triggered
- Detection completed or failed
- Error messages for debugging and troubleshooting

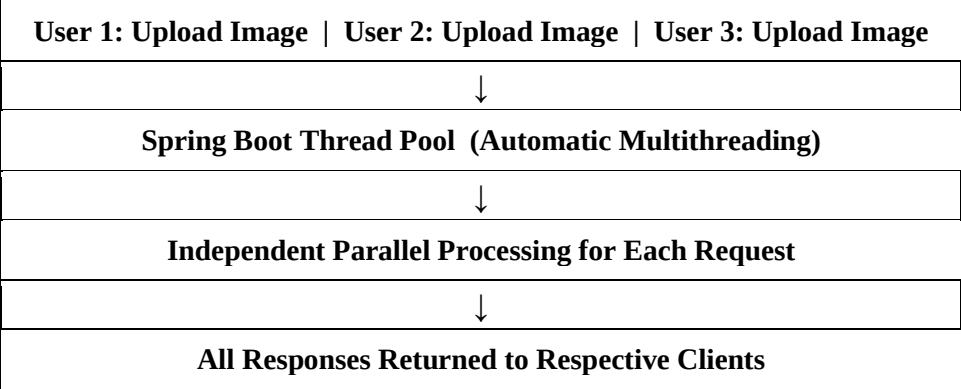
Sample Log Output

```
[INFO]  API request received - POST /upload
[INFO]  Image uploaded successfully - vehicle1.jpg
[INFO]  Detection model triggered
[INFO]  Detection completed - Plate detected: KL07AB1234
```

8. Parallel Processing and Concurrency

Spring Boot supports automatic multithreading, enabling the system to handle multiple simultaneous image upload requests without any performance degradation. Each request is processed independently in its own thread, ensuring no delays even under high load.

Concurrent Request Handling Flow



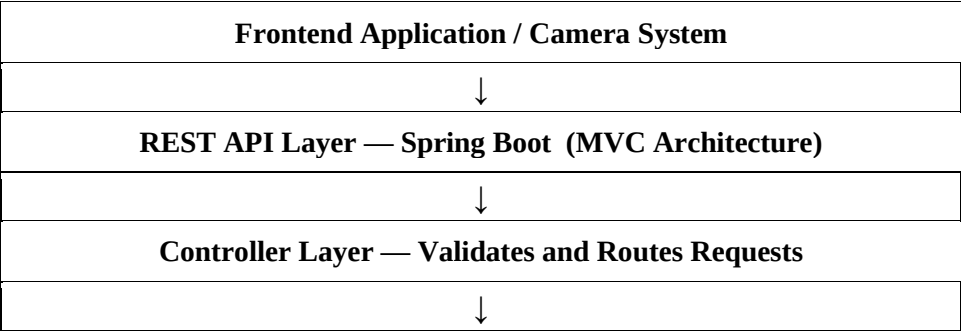
9. API Response to Client

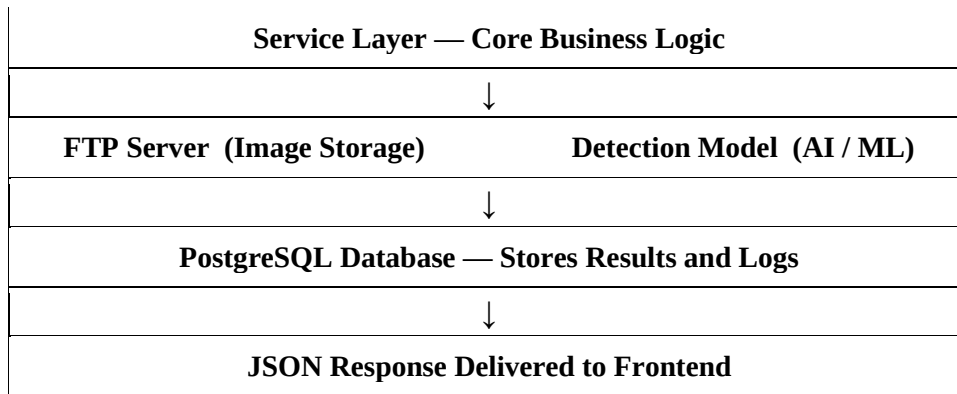
After the detection process is completed, the backend sends a structured JSON response back to the frontend containing the following information:

```
{
  "filename"      : "vehicle1.jpg",
  "plate_number"  : "KL07AB1234",
  "status"        : "success",
  "upload_time"   : "2026-03-06 10:21:30"
}
```

10. Complete System Architecture

The following flow represents the complete end-to-end architecture of the Number Plate Detection backend system:





11. Deployment

The complete backend application is packaged as an executable JAR file, making it easy to deploy and run across different environments. The application is started using a single Java command:

```
java -jar number-plate-detection.jar
```

Upon successful startup, the following services are initialized in sequence:

- Spring Boot application server starts and becomes active
- Database connection to PostgreSQL is established
- FTP server connection is configured and verified
- All REST API endpoints become available for incoming requests